



**COMPOSABLE
SECURITY**

REPORT

Security review for Lido

Prepared by: Composable Security

Report ID: LDO-af6d118a

Test time period: 2026-08-31 - 2026-09-04

Retest time period: 2026-09-08 - 2026-09-09

Report date: 2026-09-09

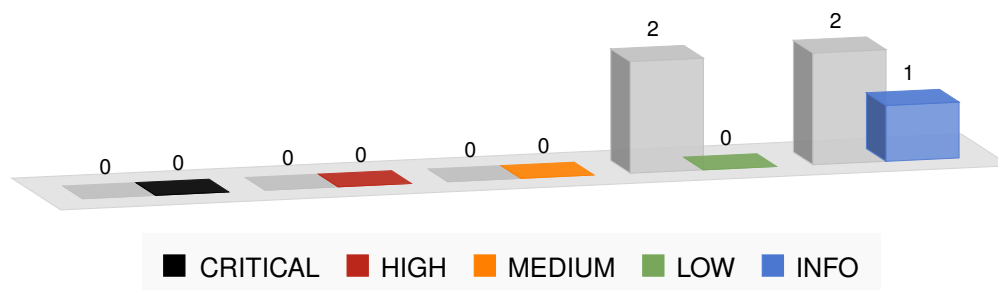
Version: 1.1

Visit: composable-security.com

Contents

1. Retest summary (2026-09-09)	2
1.1 Results	2
1.2 Scope	2
2. Current findings status	3
3. Security review summary (2026-09-04)	4
3.1 Client project	4
3.2 Results	4
3.3 Scope	5
4. Project details	6
4.1 Projects goal	6
4.2 Agreed scope of tests	6
4.3 Threat analysis	6
4.4 Testing methodology	7
4.5 Deployments	7
4.6 Disclaimer	7
5. Vulnerabilities	8
[LDO-af6d118a-L01] Stale signer used for the extra data transaction	8
[LDO-af6d118a-L02] Unconfirmed telemetry transaction reported as sent	9
6. Recommendations	12
[LDO-af6d118a-R01] Unreachable branch in the submit delay calculation	12
[LDO-af6d118a-R02] Misleading warning about a pending key rotation	13
7. Impact on risk classification	15
8. Long-term best practices	16
8.1 Use automated tools to scan your code regularly	16
8.2 Perform threat modeling	16
8.3 Use Smart Contract Security Verification Standard	16
8.4 Discuss audit reports and learn from them	16
8.5 Monitor your and similar contracts	16

1. Retest summary (2026-09-09)



The description of the current status for each retested vulnerability and recommendation has been added in its section.

1.1. Results

The **Composable Security** team participated in a one-time iteration to verify if the vulnerabilities detected during the tests (between 2026-08-31 and 2026-09-04) were correctly removed and no longer appear in the code.

The current status of detected issues is as follows:

- **Both** vulnerabilities with a **low** impact on risk have been fixed.
- **Two** security **recommendations** were handled as follows:
 - 1 has been implemented,
 - 1 has been acknowledged.

1.2. Scope

The retest scope included the same contracts, on a different commit in the same repository.

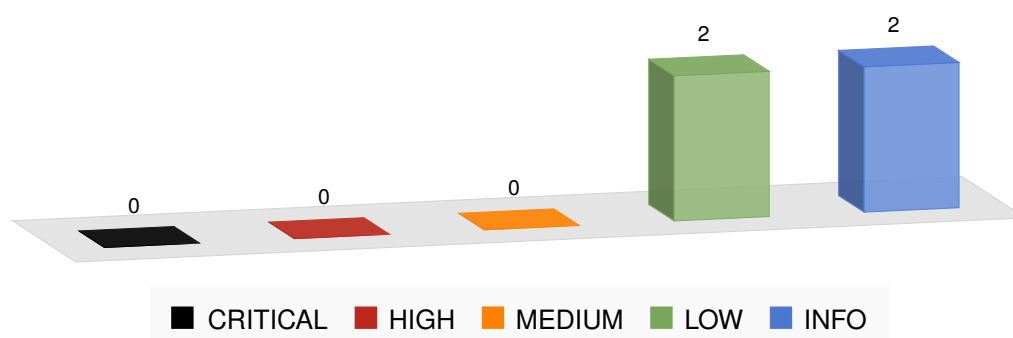
GitHub repository: <https://github.com/lidofinance/lido-oracle>

CommitID: 032c228c767759e67da43e6c40fa81732257879d

2. Current findings status

ID	Severity	Vulnerability	Status
LDO-af6d118a-L01	LOW	Stale signer used for the extra data transaction	FIXED
LDO-af6d118a-L02	LOW	Unconfirmed telemetry transaction reported as sent	FIXED
ID	Severity	Recommendation	Status
LDO-af6d118a-R01	INFO	Unreachable branch in the submit delay calculation	ACKNOWLEDGED
LDO-af6d118a-R02	INFO	Misleading warning about a pending key rotation	IMPLEMENTED

3. Security review summary (2026-09-04)



3.1. Client project

Lido is the largest decentralized staking protocol that allows users to stake ETH and receive liquid staking derivative in return. The project successfully manages over \$24B TVL supporting Ethereum growth and security since 2020.

The **Lido Oracle** project is a daemon that monitors the Lido staking protocol across the Execution and Consensus layers and submits regular update reports to Lido smart contracts. It is responsible for aggregating, verifying, and relaying critical operational data that support the protocol's decentralized staking mechanism.

In this engagement, the Composable Security team focused on **changes introduced in version 8.1** of Lido Oracle, including support for the Execution Delegation Framework (EDF).

3.2. Results

The **Lido** engaged Composable Security to review the security of **Lido Oracle - V8.1**. Composable Security conducted this assessment on 1 person-week with 2 engineers.

The summary of findings is as follows:

- No vulnerabilities with **critical**, **high** or **medium** impact on risk were identified.
- **Two** vulnerabilities with a **low** impact on risk were identified.
- Two **recommendations** have been proposed that can improve overall security and help implement best practice.
- The team was engaged and the communication was very good.

Composable Security recommends that **Lido** complete the following:

- Address all reported issues.

- Extend unit tests with scenarios that cover detected vulnerabilities where possible.
- Consider whether detected vulnerabilities may exist in other places (or ongoing projects) that have not been detected during the engagement.
- Review dependencies and upgrade to the latest versions.

3.3. Scope

The scope of the tests included selected files - used to build the image for production use - from the following repository.

GitHub repository: <https://github.com/lidofinance/lido-oracle>

CommitID: af6d118a91095585ba35d04e37656e195c5b9350

The detailed scope of tests can be found in Agreed scope of tests.

4. Project details

4.1. Projects goal

The Composable Security team focused during this security review on the following:

- Perform a customized threat analysis.
- Ensure that source code is written according to security best practices.
- Identify security issues and potential threats both for **Lido** and their users.
- The secondary goal is to improve the clarity of the code and optimize the code where possible.

4.2. Agreed scope of tests

The scope of the tests included selected files - used to build the image for production use - from the the **Lido** repository.

GitHub repository: <https://github.com/lidofinance/lido-oracle>

CommitID: af6d118a91095585ba35d04e37656e195c5b9350

Documentation:

- Audit Scope LIP-37: Execution Delegation Framework

4.3. Threat analysis

This section summarizes the potential threats that were identified during the initial threat modeling performed prior to the security review. The tests were focused, but not limited to, finding security issues that could be exploited to achieve these threats.

Key assets that require protection:

- Data in the report.
- Protocols availability.

Potential attackers goals:

- Manipulation of the data submitted.
- Inability to submit report via EDF or using EOA.
- Generation of different report hashes by oracle members.
- Denial of service for Lido Oracle or EDF.

Potential scenarios to achieve the indicated attacker's goals:

- Impersonating other members using their delegation contract.
- Getting more oracle votes due to the multiple presence of member-controller addresses.
- Recover of the oracle hot key from `MEMBER_PRIV_KEY`, `MEMBER_PRIV_KEY_2` or their `_FILE`

variants.

- An attacker who controls one of the configured execution or consensus endpoints serves a forged beacon state, validator balances or withdrawal vault state.
- The delegate is rotated while a reporting cycle is in flight.
- An attacker drains or freezes the ETH balance of the oracle account, or of the account currently registered as the delegation contract's delegate, so no report transaction can pay for gas before the frame deadline.
- A report large enough to exceed `TX_GAS_LIMIT` is silently clamped by the gas estimator, which only logs a warning, so the submission is broadcast with insufficient gas.
- Influence or bypass the business logic of the system.

4.4. Testing methodology

Security review was performed using the following methods:

- Q&A sessions with the **Lido** development team to thoroughly understand intentions and assumptions of the project.
- Initial threat modeling to identify key areas and focus on covering the most relevant scenarios based on real threats.
- Automatic tests using internal tools.
- **Manual review of the code.**

4.5. Deployments

After the security review conducted, we verified that the Dockerfile used to build an image uses the reviewed source code.

The published image with the manifest digest

sha256:ea496996214d309a6fcf9e8231627b1486cf1f92439571a1773b16fc3f2af9f4

corresponds to the commitID: **032c228c767759e67da43e6c40fa81732257879d**.

4.6. Disclaimer

Security review **IS NOT A SECURITY WARRANTY**.

During the tests, the Composable Security team makes every effort to detect any occurring problems and help to address them. However, it is not allowed to treat the report as a security certificate and assume that the project does not contain any vulnerabilities. Securing complex platforms is a multi-stage process, starting from threat modeling, through development based on best practices, security reviews and formal verification, ending with constant monitoring and incident response.

Therefore, we encourage the implementation of security mechanisms at all stages of development and maintenance.

5. Vulnerabilities

[LDO-af6d118a-L01] Stale signer used for the extra data transaction

LOW FIXED

Retest (2026-09-08)

The vulnerability has been removed as recommended.

Occurrences

- accounting.py (#L114-L125, #L127-L137)

Description

The third phase of the accounting report is sent by the `process_extra_data` method. Before submitting, the module calls `_get_slot_delay_before_data_submit` and sleeps for the number of slots returned. With the default `SUBMIT_DATA_DELAY_IN_SLOTS` of 6 this is between 6 slots and `members` * 6 slots, roughly 1 to 11 minutes for a committee of nine members. After the sleep the method refreshes the blockstamp, re-checks `can_submit_extra_data`, and calls `_submit_extra_data`, which passes the transaction to `check_and_send_transaction`.

The signer is not refreshed alongside the blockstamp. The `check_and_send_transaction` method reads `self.w3.signer.active_signer` and `self.w3.signer.is_delegated` at send time, and those two fields are assigned only by `SignerModule.process_members`, which the third phase never calls. Their values still come from the last `_get_latest_data` call made during the second phase, before the sleep.

The second phase does not have this gap: `_process_report_data` enters `_get_latest_data` again after every slept slot (consensus.py#L404-L414), so a member or delegate change is picked up within one slot.

As a result, a `HashConsensus` member change or a delegate rotation on the delegation contract that is enacted during the sleep is invisible to the third phase. With delegation enabled the transaction is still wrapped by `wrap_call_for_delegation` and signed by an account the contract no longer accepts as its delegate.

Vulnerable scenario

The following steps lead to the described result:

- ① The oracle runs with a delegation contract configured. During the second phase, the `_get_latest_data` function calls `process_members`, finds the delegation contract in the member list, and sets `active_signer` to the configured account currently returned by `getDelegate()`, with `is_delegated` set to `True`.
- ② The main data is submitted and `execute_module` calls `process_extra_data`. The method computes `slots_to_sleep` as 18 and sleeps for 216 seconds.
- ③ During the sleep the nominated hot key finishes its cooldown period, and the rotation is enacted on-chain.
- ④ The sleep ends. The module fetches a new blockstamp and checks again using the `can_submit_extra_data` function, which still passes because the extra data is genuinely outstanding. Neither check touches the signer, so `active_signer` remains the account resolved in step 1.
- ⑤ `_submit_extra_data` calls `check_and_send_transaction`, which wraps the call through the delegation contract and builds it with the stale account. The static call in `_check_transaction` reverts, because that account is no longer the delegate, so the method logs `Transaction reverted`. and returns without broadcasting anything.

Result: The extra data submission is skipped for the whole cycle and the report stays in its third phase until a later cycle re-resolves the signer.

Recommendation

Refresh the signer after the sleep the same way the second phase does, by calling `_get_latest_data` instead of `_get_latest_blockstamp`:

```
sleep(seconds_to_sleep)

latest_blockstamp, _ = self._get_latest_data()
if not self.can_submit_extra_data(latest_blockstamp):
    logger.info({'msg': 'Extra data can not be submitted.'})
    return
```

References

1. SCSVS G4: Business logic

[LDO-af6d118a-L02] Unconfirmed telemetry transaction reported as sent

LOW FIXED

Retest (2026-09-08)

The vulnerability has been removed. Now, when the deadline passes, function returns `None` instead of `tx_hash` when it is set (otherwise it raises an error as it did previously). There is no final check for transaction existence, because the Lido team decided not to add one more RPC call for this. Nobody uses the return value, no retry either way, only the log line differs and it has the tx hash.

Occurrences

- `telemetry_data_bus.py` (#L169-L170, #L126-L131)

Description

The `_send_telemetry` method broadcasts the `DataBus` telemetry transaction and then polls until `TELEMETRY_TX_SEND_TIMEOUT_SECONDS` (4 minutes by default) elapses. Inside the loop the inclusion check is explicit: `w3.eth.get_transaction` is called for the stored hash and the hash is returned only when `blockNumber` is not `None` (line 150). Once the deadline passes, the loop exits and the trailing `if tx_hash: return tx_hash` returns the very same hash with that check omitted.

At that point the only established fact about `tx_hash` is that `send_raw_transaction` accepted it. The transaction may still be pending, or may have been dropped from the mempool. A caller cannot tell the two apart, because a mined transaction and an unmined one produce an identical return value, and `send_telemetry` goes on to log `DataBus telemetry sent` with the hash and return normally.

The mismatch propagates into the report flow. The `_try_send_telemetry` method treats any call that does not raise as a success, and `_send_report_telemetry` records `self._last_sent_report_hash = report_hash` based on that apparent success, even if the transaction has not been included in a block. Because `_last_sent_report_hash` is instance state on the module, every later cycle for the same report hash takes the duplicate-check early return, so the telemetry for that report is never retried while the process lives.

Vulnerable scenario

The following steps lead to the described result:

- ① A report finishes and `_send_report_telemetry` calls `send_telemetry` with the event (`ORACLE_REPORT`) and the report payload.
- ② On the first loop iteration `tx_hash` is `None`, so `_send_telemetry` broadcasts the transaction, stores its hash, and records the nonce it used.
- ③ The `DataBus` chain is congested and the transaction stays pending.

On every following iteration `build_transaction_params` derives the nonce from `w3.eth.get_transaction_count` at the latest block, which does not count the pending transaction, so `params.get('nonce')== nonce` holds, the iteration sleeps 10 seconds and continues, and the inclusion check at line 150 is never reached.

- ④ After 240 seconds the `while time.monotonic() < deadline` condition fails and the loop exits with `tx_hash` still set.
- ⑤ The trailing `if tx_hash` returns the unmined hash, `send_telemetry` logs `DataBus telemetry sent.`, and `_try_send_telemetry` returns `True`.
- ⑥ `_send_report_telemetry` sets `_last_sent_report_hash` to this report hash, so every later cycle for the same report returns at the duplicate check without attempting another send.

Result: A pending or dropped telemetry transaction is recorded as successfully sent and that report's telemetry is never re-attempted by the running process.

Recommendation

Apply the same inclusion check after the deadline that the loop applies inside it, and keep the timeout error for the unconfirmed case:

```
if tx_hash:
    with suppress(TransactionNotFound):
        sent_tx = w3.eth.get_transaction(HexBytes(tx_hash))
        if sent_tx.get('blockNumber') is not None:
            return tx_hash

    raise TelemetrySendTimeoutError(
        f"Timed out sending DataBus telemetry transaction after {
            variables.TELEMTRY_TX_SEND_TIMEOUT_SECONDS}s."
    )
```

The `w3.eth.get_transaction` call raises `TransactionNotFound` for a dropped transaction, so the lookup has to tolerate that and fall through to the raise. Raising makes `_try_send_telemetry` return `False`, which leaves `_last_sent_report_hash` unset and allows the next cycle to retry the send.

References

1. SCSVS G4: Business logic
2. SCSVS G6: Communications

6. Recommendations

[LDO-af6d118a-R01] Unreachable branch in the submit delay calculation

INFO

ACKNOWLEDGED

Retest (2026-09-08)

The recommendation has not been implemented.

Lido's team response: *The operand is logically redundant, but it narrows the type for pyright before members.index, so removing it would need an extra check in the same place.*

Description

The `_get_slot_delay_before_data_submit` method returns a zero delay for members that hold the submit data role, and guards the same early return with a second condition on the reporting address:

```
member = self.get_member_info(blockstamp)
report_address = self.w3.signer.hash_consensus_member_address

if member.is_submit_member or report_address is None:
    return 0
```

The second operand can never decide the branch. In `get_member_info`, `is_submit_member` is initialised to `True` and reassigned only inside `if hash_consensus_member_address and active_signer:`.

- When `hash_consensus_member_address` is `None` that branch is skipped, the function `is_submit_member` keeps its `True` default, and the first operand already short-circuits the `or`.
- When the address is not `None`, `report_address`, read from the same property, is not `None` either. Dry mode, where no account is configured at all, follows the first of those two paths.

The redundant operand also obscures what the code depends on. The `members.index(report_address)` call would raise `ValueError` for a `None` address, so a reader can mistake the guard for the protection against that, when the actual protection is the `True` default of `is_submit_member`. If that default ever changes, the guard silently turns a missing signer into a zero delay, which drops the fast lane stagger and makes the member submit in the

same slot as everyone else.

Here is the place where the redundant condition can be removed:

- consensus.py#L495-L499

Recommendation

Drop the unreachable operand and read the reporting address only after the early return, so the remaining code states plainly that it requires an address:

```
member = self.get_member_info(blockstamp)

if member.is_submit_member:
    return 0

report_address = self.w3.signer.hash_consensus_member_address
```

References

1. SCSVS G11: Code clarity
2. SCSVS G4: Business logic

[LDO-af6d118a-R02] Misleading warning about a pending key rotation

INFO

IMPLEMENTED

Retest (2026-09-08)

The recommendation has been implemented. The warning has been reworded to describe the observed condition and the address and block hash have been added to the log.

Description

The `get_member_info` method emits a warning when the resolved reporting address is neither a HashConsensus member nor a holder of the submit data role. The message attributes the situation to a pending key rotation and tells the operator that the oracle will keep polling and pick the change up automatically once it is enacted on-chain. That explanation does not match the condition that produced it.

The branch sits inside `if hash_consensus_member_address and active_signer:`, and both values come from `SignerModule.process_members`, which sets `active_signer` only after

matching a configured account against the current member list, either through the delegation contract's delegate or through a member address directly. A genuinely pending rotation, where the operator has configured the new key while the member set still holds the old address, never reaches this branch: `process_members` finds no match, leaves `active_signer` as `None`, and logs `None of the configured accounts is an active member`. instead.

The branch is also unreachable on the main path. In `_get_latest_data` the member list and `get_consensus_state_for_member` are read at the same `latest_blockstamp.block_hash`, so an address that `process_members` just matched cannot come back as a non-member.

It becomes reachable only where `get_member_info` is called with a newer blockstamp than the one used for the last `process_members` call, for instance the accounting third phase, where `_get_slot_delay_before_data_submit` is handed a freshly fetched blockstamp. In that case the cause is a member-set change between resolution and the query, so the log entry states the opposite of what the operator needs: the address has been removed, not pending.

Here is the place where the message needs correcting:

- `consensus.py#L200-L207`

Recommendation

State the observed condition in the warning message rather than a presumed cause. Consider adding the resolved address and the block hash in the log entry to make it actionable, since the operator can read the member list at that exact block and confirm whether the address was removed.

References

1. SCSVS G11: Code clarity

7. Impact on risk classification

Risk classification is based on the one developed by OWASP ¹, however, it has been adapted to the immutable and transparent code nature of Web3 projects. The Web3 ecosystem forgives far fewer mistakes than in the case of traditional applications, the servers of which can be covered by many layers of security.

Therefore, the classification is more strict and indicates higher priorities for paying attention to security.

OVERALL RISK SEVERITY				
Impact on risk	HIGH	CRITICAL	HIGH	MEDIUM
	MEDIUM	MEDIUM	MEDIUM	LOW
	LOW	LOW	LOW	INFO
	Likelihood			
		HIGH	MEDIUM	LOW

¹OWASP Risk Rating methodology

8. Long-term best practices

8.1. Use automated tools to scan your code regularly

It's a good idea to incorporate automated tools into the code writing process. This will allow basic security issues to be detected and addressed at a very early stage.

8.2. Perform threat modeling

Before implementing or introducing changes, perform threat modeling and think with your team about what can go wrong. Set potential targets of the attacker and possible ways to achieve them, keep it in mind during implementation to prevent bad design decisions.

8.3. Use Smart Contract Security Verification Standard

Use proven standards to maintain a high level of security for your contracts. Treat individual categories as checklists to verify the security of individual components. Expand your unit tests with selected checks from the list to be sure when introducing changes that they did not affect the security of the project.

8.4. Discuss audit reports and learn from them

The best guarantee of security is the constant development of team knowledge. To use the audit as effectively as possible, make sure that everyone in the team understands the mistakes made. Consider whether the detected vulnerabilities may exist in other places, audits always have a limited time and the developers know the code best.

8.5. Monitor your and similar contracts

Use the tools available on the market to monitor key contracts (e.g. the ones where user's tokens are kept). If you have used code from another project, monitor their contracts as well and introduce procedures to capture information about detected vulnerabilities in their code.



Damian Rusinek

Smart Contracts Auditor

@drdr_zz

damian.rusinek@composable-security.com



Paweł Kuryłowicz

Smart Contracts Auditor

@wh01s7

pawel.kurylowicz@composable-security.com

